FINDING TIME COMPLEXITY OF ALGORITHMS

Problem 1: Finding Complexity using Counter Method

```
#include <stdio.h>

int main() {

int n;

if (scanf("%d", &n) != 1) return 0;

int count = 0;

int i = 1;

count++;

int s = 1;

count++;

while (++count && s <= n) {

i++;

count++;

s += i;

count++;

}

printf("%d\n", count);

return 0
```

```
}
```

Problem 2: Finding Complexity using Counter method

```
#include <stdio.h>

void func(int n) {
    int count = 0;
     count++;
    if(n == 1) {
       count++;
    }
    else {
       for(int i = 1; i <= n; i++) {
          count++;
      for(int j = 1; j <= n; j++) {
            count++;
                  count++;
            count++;
```

```
            count++;
            break;
        }
    }
    count++;
  }

  printf("%d\n", count);
}

int main() {
  int n;
  if (scanf("%d", &n) == 1 && n > 0) {
    func(n);
  }
```

```
    return 0;


}
```

Problem 3: Finding Complexity using Counter Method

```
#include <stdio.h>

int main()

{

int n, i;

int count = 0;

scanf("%d", &n);

for (i = 1; i <= n; ++i)

{

count++;

if (n % i == 0)

{

count++;

}

count++;

}

count++;

printf("%d", count);

return 0;
```

```
}
```

Problem 4: Finding Complexity using Counter Method

```
#include <stdio.h>

void function(int n)

{

int c = 0;

int i = n / 2;

c++;

for (; (c++, i < n); i++)

{

for (int j = 1; (c++, j < n); j = 2 * j)

{

for (int k = 1; (c++, k < n); k = k * 2)

{

c++;

}

}

}

printf("%d", c);
```

```
}

int main()

{

int n;

scanf("%d", &n);

function(n);

return 0;

}
```

Problem 5: Finding Complexity using counter method

```
#include <stdio.h>

void reverse(int n)

{

int rev = 0, remainder;

int c = 2;

while (n != 0)

{

remainder = n % 10;

c++;

rev = rev * 10 + remainder;
```

```
c++;

n /= 10;

c++;

c++;

}

c++;

printf("%d", c);

}

int main()

{

int n;

scanf("%d", &n);

reverse(n);

return 0;

}
```

GREEDY ALGORITHMS

1-G-Coin Problem

```
#include <stdio.h>
int main() {
```

```
int amount;

if (scanf("%d", &amount) != 1) return 0;

int denominations[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1}; int

n = sizeof(denominations) / sizeof(denominations[0]); int

total_count = 0;

for (int i = 0; i < n; i++) {

if (amount == 0) {

break;

}

total_count += amount / denominations[i];

amount %= denominations[i];

}

printf("%d\n", total_count);

return 0;

}
```

2-G-Cookies Problem

```
#include <stdio.h>

#include <stdlib.h>
int compare(const void *a, const void *b) {
```

```
return (*(int *)a - *(int *)b);

}

int main() {

int n;

if (scanf("%d", &n) != 1) return 0;

int *g = (int *)malloc(n * sizeof(int));

for (int i = 0; i < n; i++) {

scanf("%d", &g[i]);

}

int m;

if (scanf("%d", &m) != 1) return 0;

int *s = (int *)malloc(m * sizeof(int));

for (int j = 0; j < m; j++) {

scanf("%d", &s[j]);

}

qsort(g, n, sizeof(int), compare);

qsort(s, m, sizeof(int), compare);

int child_ptr = 0;

int cookie_ptr = 0;
```

```
while (child_ptr < n && cookie_ptr < m) {

if (s[cookie_ptr] >= g[child_ptr]) {

child_ptr++;

}

cookie_ptr++;

}

printf("%d\n", child_ptr);

free(g);

free(s);

return 0;

}
```

3-G-Burger Problem

```
#include <stdio.h>

#include<math.h>

int main()

{

int n;

scanf("%d",&n);

int arr[n];
```

```
for(int i=0;i<n;i++)
{
scanf("%d",&arr[i]);
}
for(int i=0;i<n-1;i++)
{
for(int j=0;j<n-i-1;j++)
{
if(arr[j]<arr[j+1])
{
int temp=arr[j];
arr[j]=arr[j+1];
arr[j+1]=temp;
}
}
}
int s=0;
for(int i=0;i<n;i++)
{
```

```
for (int i = 0; i < n; i++) {

}

printf("%d",s);

}
```

4-G-Array Sum max problem

```
#include <stdio.h>

#include <stdlib.h>



int cmp(const void *a, const void

*b) {

   return (*(int *)a > *(int *)b) -

(*(int *)a < *(int *)b);

}



int main() {

if (int_a > int_b) return 1;
```

```
for (int i = 0; i < n; i++) {
    int n;

    scanf("%d", &n);




    int a[n];

    for (int i = 0; i < n; i++)

        scanf("%d", &a[i]);




    qsort(a, n, sizeof(int), cmp);




    long long sum = 0;

    for (int i = 0; i < n; i++)

        sum += (long long)a[i] * i;




    printf("%lld", sum);
if (int_a > int_b) return 1;
```

```
for (int i = 0; i < n; i++) {


   return 0;

}
```

5-G-Product of Array elements-Minimum #include

```
<stdio.h>

#include <stdlib.h>

int compare_asc(const void *a, const void *b) {

long long int_a = *(long long *)a;

long long int_b = *(long long *)b;
```

```
if (int_a > int_b) return 1;
```

```
if (int_a < int_b) return -1;

return 0;

}

int main() {

int n;

if (scanf("%d", &n) != 1) return 0;

long long *a = (long long *)malloc(n * sizeof(long long));

long long *b = (long long *)malloc(n * sizeof(long long));

if (!a || !b) return 0;

for (int i = 0; i < n; i++) {

scanf("%lld", &a[i]);

}

for (int i = 0; i < n; i++) {

scanf("%lld", &b[i]);

}

qsort(a, n, sizeof(long long), compare_asc);

qsort(b, n, sizeof(long long), compare_asc);

long long min_sum = 0;

for (int i = 0; i < n; i++) {
```

```
min_sum += a[i] * b[n - 1 - i];

}

printf("%lld\n", min_sum);

free(a);

free(b);

return 0;

}
```

DIVIDE AND CONQUER

1-DP-Playing with Numbers

```
#include <stdio.h>

int main() {

int n;

scanf("%d", &n);

long long dp[n + 1];

dp[0] = 1;

for (int i = 1; i <= n; i++) {

dp[i] = dp[i - 1];

if (i >= 3) {

dp[i] += dp[i - 3];
```

```
}

}

printf("%lld", dp[n]);

return 0;

}
```

3-DP-Longest Common Subsequence #include

```
<stdio.h>

#include <string.h>

int main() {

char s1[1000], s2[1000];

scanf("%s", s1);

scanf("%s", s2);

int n = strlen(s1);

int m = strlen(s2);

int dp[n + 1][m + 1];

for (int i = 0; i <= n; i++) {

for (int j = 0; j <= m; j++) {

if (i == 0 || j == 0) {

dp[i][j] = 0;
```

```
}
else if (s1[i - 1] == s2[j - 1]) {
dp[i][j] = dp[i - 1][j - 1] + 1;
}
else {
dp[i][j] = dp[i - 1][j] > dp[i][j - 1]
? dp[i - 1][j]
: dp[i][j - 1];
}
}
}
printf("%d", dp[n][m]);
return 0;
}
```

4-DP-Longest non-decreasing Subsequence

```
#include <stdio.h>
int main() {
int n;
scanf("%d", &n);
```

```
int a[n];

int dp[n];

for (int i = 0; i < n; i++) {

scanf("%d", &a[i]);

dp[i] = 1;

}

int ans = 1;

for (int i = 1; i < n; i++) {

for (int j = 0; j < i; j++) {

if (a[j] <= a[i] && dp[j] + 1 > dp[i]) {

dp[i] = dp[j] + 1;

}

}

if (dp[i] > ans)

ans = dp[i];

}

printf("%d", ans);

return 0;

}
```

COMPETITIVE PROGRAMMING

1- Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity

```
#include <stdio.h>

int main() {

int n;

scanf("%d", &n);

int count[n + 1];

for (int i = 0; i <= n; i++)

count[i] = 0;

for (int i = 0; i < n; i++) {

int x;

scanf("%d", &x);

count[x]++;

if (count[x] == 2) {

printf("%d", x);

return 0;

}

}

return 0;
```

```
}
```

2- Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity

```
#include <stdio.h>

int main() {

int n;

scanf("%d", &n);

int a[n];

for (int i = 0; i < n; i++)

scanf("%d", &a[i]);

for (int i = 0; i < n; i++) {

for (int j = i + 1; j < n; j++) {

if (a[i] == a[j]) {

printf("%d",

a[i]); return 0;

}

}

}

return 0;
}
```

3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity

```
#include <stdio.h>

int main() {

int T;

scanf("%d", &T);

while (T--) {

int n1, n2;

scanf("%d", &n1);

int a[n1];

for (int i = 0; i < n1; i++)

scanf("%d", &a[i]);

scanf("%d", &n2);

int b[n2];

for (int i = 0; i < n2; i++)

scanf("%d", &b[i]);

int i = 0, j = 0;

int first = 1;

while (i < n1 && j < n2) {
if (a[i] == b[j]) {
```

```
if (!first)

printf(" ");

printf("%d",

a[i]); first = 0;

i++;

j++;

}

else if (a[i] < b[j]) {

i++;

}

else {

j++;

}

}

printf("\n");

}

return 0;

}
```

4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity

```
#include <stdio.h>

int main() {

int T;

scanf("%d", &T);

while (T--) {

int n1, n2;

scanf("%d", &n1);

int a[n1];

for (int i = 0; i < n1; i++)

scanf("%d", &a[i]);

scanf("%d", &n2);

int b[n2];

for (int i = 0; i < n2; i++)

scanf("%d", &b[i]);

int i = 0, j = 0;

int first = 1;

while (i < n1 && j < n2) {

if (a[i] == b[j]) {
if (!first)
```

```
printf(" ");

printf("%d",

a[i]); first = 0;

i++;

j++;

}

else if (a[i] < b[j]) {

i++;

}

else {

j++;

}

}

printf("\n");

}

return 0;

}
```

5- Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity

```
#include <stdio.h>
```

```
int main() {

int n, k;

scanf("%d", &n);

int a[n];

for (int i = 0; i < n; i++)

scanf("%d", &a[i]);

scanf("%d", &k);

int i = 0, j = 1;

while (i < n && j < n) {

if (i == j) {

j++;

}

else if (a[j] - a[i] == k) {

printf("1");

return 0;

}

else if (a[j] - a[i] < k) {

j++;

}
```

```
else {

i++;

}

}

printf("0");

return 0;

}
```

6- Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

```
#include <stdio.h>

int main() {

int n, k;

scanf("%d", &n);

int a[n];

for (int i = 0; i < n; i++) {

scanf("%d", &a[i]);

}

scanf("%d", &k);

int i = 0;

int j = 1;
```

```
while (i < n && j < n) {
if (i == j) {
j++;
continue;
}
int diff = a[j] - a[i];
if (diff == k) {
printf("1");
return 0;
}
if (diff < k)
j++;
else
i++;
}
printf("0");
return 0;
}
```